

Auditoría Caudalímetros

Usuario: [File: PROMPT_AUDITORIA_RONDA6_CIERRE_SUBHITO_2_2.md]

revisa esto con detalle, necesito lograr este objetivo

use.ai: # Dictamen de auditoría, Ronda 6: Subhito 2.2 (caudalímetros y firmware)

Firmware V4.1 tal como está: RECHAZADO. Con el firmware que propongo abajo y las dos pruebas de banco del protocolo: APROBADO CON OBSERVACIONES.

El filtro de coherencia V4.1 no puede detectar el fantasma que se midió. El blanking de 1500 μ s deja **un solo pulso por ráfaga**. Lo que llega al filtro de dispersión es entonces un tren casi periódico, con $\frac{dt_{\max}}{dt_{\min}} \approx 1$. El filtro lo acepta como si fuera una turbina girando.

Este rechazo no exige tocar el hardware. La mejora de firmware que sí puede funcionar es otra: **validar el ancho de cada pulso** (duración real del nivel LOW y HIGH). Un pulso de turbina dura milisegundos. Un glitch de conmutación dura microsegundos. Queda una única barrera física posible: que el ruido produzca niveles LOW de más de unos 0,8 ms. En ese caso ningún firmware lo separa y habrá que corregir masa o cableado. El firmware propuesto incluye los contadores que permiten saber en cuál de los dos casos están.

Pregunta 1: ¿Basta el filtro de coherencia temporal?

No. Hay tres motivos.

a) El filtro está desactivado justo en el caso medido. El blanking acepta el primer flanco que llega pasados 1500 μ s desde el último aceptado. Si el ruido llega en ráfagas, cada ráfaga aporta 1 pulso y los intervalos entre pulsos aceptados son los intervalos entre ráfagas. Esas ráfagas están ligadas a la cinemática del motor (la frecuencia fantasma crece con las RPM), así que son regulares:

$$\frac{dt_{\max}}{dt_{\min}} \approx 1 \leq 4 \rightarrow \text{se acepta como flujo real}$$

b) Un ruido sincronizado con los pasos también pasa. Si cada flanco de paso induce un glitch, el periodo de ruido es $T_s = 60/(3200 \cdot \text{RPM})$. Los pulsos aceptados quedan separados por $\lceil 1500/T_s \rceil \cdot T_s$, que es un valor constante. Ejemplo a 60 RPM: $T_s = 312,5 \mu$ s, se acepta 1 de cada 5 y queda una frecuencia de 640 Hz con dispersión exactamente 1,0. Eso da $Q = 640,000/687,33 = 931$ mL/min, por debajo del límite de plausibilidad de 2500 mL/min, así que se acepta.

c) La banda muerta de 30 mL/min no protege.

RPM	Fantasma	Q con K=687,33	Q con K=196,5
60	17 Hz	24,7 → cortado	86,5 → pasa
90	55 Hz	80,0 → pasa	280 → pasa

La banda muerta además elimina permeado real de 20–30 mL/min, que está dentro del rango esperado. Pierden sensibilidad y no ganan inmunidad.

Escenario que engaña al algoritmo: cualquier ruido de fase fija respecto de los pasos o las fases del motor, que es exactamente lo que observaron. Hay un fallo más que nadie detectaría: un filtro que rechazara todo también pasaría la "Prueba de Permeado Seco". **La prueba en seco mide especificidad, no sensibilidad.** Hace falta una prueba positiva complementaria (ver Pregunta 5).

Pregunta 2: Red RC (4,7 kΩ + 200 nF)

$$\tau = 0,94 \text{ ms}, \text{ } f_c = \frac{1}{2\pi\tau} \approx 169 \text{ Hz}$$

Asimetría. La bajada es rápida porque el transistor del sensor descarga el capacitor. La subida es lenta porque carga a través de los 4,7 kΩ. Con los umbrales del ESP32 ($V_{IL}=0,25V_{DD}$, $V_{IH}=0,75V_{DD}$):

$$t_{V_{IL}} = -\tau \ln(0,75) = 0,27 \text{ ms}, \text{ } t_{V_{IH}} = -\tau \ln(0,25) = 1,30 \text{ ms}$$

Cada subida pasa unos **1,03 ms en la zona indefinida**. Que yo sepa, el ESP32 clásico no ofrece histéresis Schmitt configurable en los GPIO, a diferencia de variantes posteriores; conviene verificarlo en la hoja de datos. Con eso, unos cientos de mV de ruido de masa bastan para generar varias transiciones por subida. Y el blanking de 1,5 ms **es menor que el tiempo de subida completo** ($10\text{--}90\% \approx 2\tau = 2,07 \text{ ms}$). Un cruce espurio al final de la rampa puede contarse como un pulso extra.

Hipótesis a verificar. Recuerdo, sin haberlo verificado en esta revisión, que la hoja de datos de la YF-S401 da del orden de $F \approx 98 \cdot Q$ (≈ 5880 pulsos/L). Su $K = 196,5$ es casi exactamente el doble. Un doble conteo por la rampa lenta lo explicaría. La comparación ISR vs PCNT del firmware propuesto lo confirma o lo descarta.

El RC ayuda y perjudica a la vez:

Ayuda: atenúa unos -25 dB a $3,2$ kHz el ruido inyectado en el cable de señal respecto de la masa del ESP32.

No sirve contra el rebote de masa (sensor y ESP32 con potenciales de GND distintos por la corriente del DM860 en el modo de cátodo común). En ese caso el capacitor acopla el ruido en lugar de filtrarlo.

Limita la frecuencia máxima. El HIGH útil se reduce a semiperiodo $- 1,3$ ms y desaparece por encima de unos 385 Hz. En alimentación a 100 RPM (≈ 267 Hz con $K=196,5$) queda un margen de $0,57$ ms: suficiente, pero justo.

Mitigación por software: máquina de estados en ambos flancos (CHANGE), relectura del nivel real en el registro y anchos mínimos de LOW y HIGH. Es lo que implementa el código.

Pregunta 3: ¿PCNT o ISR?

El filtro de glitches del PCNT (hasta 1023 ciclos APB $\approx 12,8$ μ s) es superior a la ISR actual. La ISR se dispara con glitches de nanosegundos; el PCNT exige que el nivel se mantenga estable. Pero **$12,8$ μ s no alcanzan** para validar un pulso de turbina, que dura milisegundos, y el PCNT no da marcas de tiempo.

Mi recomendación es una arquitectura doble:

Medición principal: ISR en CHANGE con validación de ancho de LOW/HIGH (cientos de μ s a ms), periodo mínimo y mediana de periodos.

Testigo en silencio: PCNT en el mismo pin con filtro de 12 μ s, solo para diagnóstico. Si PCNT $\approx 2\times$ los pulsos válidos, el doble conteo está confirmado. Si en seco PCNT > 0 y los válidos $= 0$, el firmware está haciendo su trabajo.

Pregunta 4: ¿Sirve la turbina para medir permeado por debajo de 100 mL/min?

No como medición certificada del balance de materia. Operar fuera del rango lineal del fabricante con un K único es objetable ante un tribunal. Lo es más aún si la recta de referencia tiene ordenada al origen distinta de cero.

Referencia primaria recomendada: gravimetría, $Q = \frac{\Delta m}{\rho \cdot \Delta t}$, con HX711 o con una balanza de laboratorio y cronómetro, que no exige tocar el gabinete. La turbina queda como **indicador en línea calibrado contra la balanza**, con su incertidumbre declarada.

Procedimiento para Antonella, si se mantiene la turbina:

Línea de permeado llena y purgada de aire, con el firmware nuevo.

Cinco caudales objetivo ($\approx 20, 40, 60, 80$ y 100 mL/min), fijados con una válvula de retentado o regulando la TMP.

En cada punto, 3 repeticiones de al menos 3 min de recolección pesada (ρ a la temperatura medida). Registrar pulsos válidos acumulados ($\$N\$$), no la frecuencia instantánea.

$\$K_i = N_i/V_i\$$ en pulsos/L. Ajustar una curva $\$K(Q)\$,$ lineal o por tramos, no un K único. Informar la repetibilidad (desvío estándar entre repeticiones) y el límite de cuantificación: el caudal por debajo del cual el CV supera el 10 %.

No usar la autocalibración contra el caudal teórico de la bomba. Es circular: el mL/rev de una peristáltica cambia con la contrapresión.

Rehacer también la calibración de alimentación. La actual ($\$F = 196\{, \}50, Q + 20\{, \}98\$$) está contaminada y no debe usarse.

Pregunta 5: Dictamen y acciones por prioridad

Bloqueantes en V4.1: filtro estructuralmente ciego (P1), calibraciones contaminadas, detección de cable cortado anulada por el ruido ($n > 0$ resetea el fallo) y volumen integrado con pulsos sin validar.

Condición para la aprobación:

Integrar el firmware de abajo.

Prueba de especificidad (seco): 0 pulsos válidos y 0 mL acumulados durante 5 min a 20, 40, 60, 80 y 100 RPM. Hacerla también durante las rampas y con Wi-Fi y SCADA activos. Registrar glitches y PCNT como evidencia.

Prueba de sensibilidad (húmedo): con la misma configuración, permeado real a 20–100 mL/min contra balanza, con error $\leq \pm 5 \%$ sobre la curva $K(Q)$. Sin esta prueba, la del punto 2 no demuestra nada.

Barrera física: si en seco aparecen pulsos válidos ($LOW > 0,8 \text{ ms}$), el ruido es de masa o de nivel y no hay firmware que lo resuelva. Hay que separar las masas y llevar el retorno del sensor por par trenzado o blindado.

Recalibrar alimentación y permeado (P4).

Observaciones no bloqueantes:

`paradaEmergencia()` invocada por Wi-Fi **no es una parada de emergencia** en sentido normativo (ISO 13850 / IEC 60204-1). Hace falta un pulsador de hongo cableado. Deben describirla como "parada rápida por software con deshabilitación del driver".

El ruido a 30–40 RPM es compatible con la resonancia de media banda del híbrido NEMA 34. Conviene evitar esa banda en el diseño factorial o revisar la configuración del DM860.

El calentamiento sugiere revisar la reducción de corriente en reposo del driver (DIP de "half current").

`handleCalibrarRpmQ` usa la frecuencia de 1 s, que es ruidosa. Deberían usarse pulsos acumulados en régimen estable.

Pregunta 6: Código propuesto (reemplazo directo, ESP32 core 3.x)

La interfaz pública se mantiene; el `.ino` compila sin cambios. `pulsosBrutos()` ahora devuelve los flancos crudos que ve la ISR, y la telemetría nueva es opcional.

`caudalimetro.h`

```
#pragma once
#include <Arduino.h>
#include "config.h"

//
=====
=====
// CAUDALÍMETRO YF-S401 – V5 (Auditoría Ronda 6)
// Validación de pulso por ancho de nivel LOW/HIGH + periodo mínimo +
// mediana.
// Contador PCNT en silicio (filtro 12 µs) como testigo de diagnóstico.
//
=====
=====

#ifndef CAUDAL_USAR_PCNT_DIAG
#define CAUDAL_USAR_PCNT_DIAG 1
#endif

#if CAUDAL_USAR_PCNT_DIAG
#include "driver/pulse_cnt.h"
#endif

struct ParamPulso {
    uint32_t anchoMinLow_us;           // Duración mínima del nivel LOW de
    un pulso real
    uint32_t anchoMinHigh_us;          // Duración mínima del nivel HIGH
    previo a un flanco de bajada
    uint32_t periodoMin_us;            // Periodo mínimo admisible
    (frecuencia máxima física)
    uint32_t periodoMax_us;            // Sin pulso válido en este lapso =>
    turbina detenida
    float qMin_mLmin;                  // Límite de cuantificación (debe
```

```

float    qmin_mLmin,           // Limite de cuantificación (debe
surgir de la calibración)
float    toleranciaMediana;    // Desvío relativo admitido respecto
de la mediana (0.40 = ±40 %)
float    fraccionCoherenteMin; // Fracción mínima de periodos dentro
de tolerancia
};

class Caudalimetro {
public:
    static constexpr uint8_t N_PER = 64;

    Caudalimetro(uint8_t pin, float k, const char* nombre, bool
esAlimentacion = true);

    void begin();
    void actualizar(float dt_s, bool bombaEmpuja);

    float caudal_mLmin() const { return _q; }
    float caudal_Lmin()  const { return _q / 1000.0f; }
    float frecuencia_Hz() const { return _f; }
    float volumen_L()    const { return _vol; }
    bool  sinSenal()     const { return _fallo; }
    void  resetVolumen() { _vol = 0.0f; }
    const char* nombre() const { return _nombre; }
    bool  esAlimentacion() const { return _esAlimentacion; }

    // Diagnóstico (ventana de 1 s)
    uint32_t pulsosBrutos() const { return _vFlancos; } //
Flancos crudos vistos por la ISR
    uint32_t pulsosValidos() const { return _vValidos; } //
Pulsos con ancho y periodo válidos
    uint32_t glitchesVentana() const { return _vGlitch; } //
Rechazados por ancho
    uint32_t rapidosVentana() const { return _vRapidos; } //
Rechazados por periodo < mínimo
    int32_t pcntVentana() const { return _vPcnt; } //
Testigo en silicio (-1 si no disponible)
    float  coherencia() const { return _coherencia; } // 0..1

    // Diagnóstico acumulado
    uint32_t validosAcumulados() const { return _accValidos; }
    uint32_t glitchesAcumulados() const { return _accGlitch; }
    uint32_t ventanasRechazadas() const { return _accVentanasRech; }

    void setK(float nuevoK) { if (nuevoK > 0.1f) _k = nuevoK; }
    float getK() const { return _k; }

    void setParametros(const ParamPulso& p);
    const ParamPulso& parametros() const { return _p; }

private:
    static void IRAM_ATTR isrPuente(void* arg);
    void IRAM_ATTR isr();
    bool IRAM_ATTR leerNivel() const;
    static uint32_t mediana(uint32_t* v, uint8_t n);

    const uint8_t pin;

```

```

float      _k;
const char* _nombre;
const bool  _esAlimentacion;
ParamPulso  _p;

// Estado de la ISR
volatile bool    _nivelAlto = true;
volatile bool    _bajadaCandidata = false;
volatile uint32_t _tBajada = 0;
volatile uint32_t _tSubida = 0;
volatile bool    _hayUltValido = false;
volatile uint32_t _tUltValido = 0;

// Acumuladores de ventana (escritos por la ISR)
volatile uint32_t _flancos = 0;
volatile uint32_t _validos = 0;
volatile uint32_t _glitch = 0;
volatile uint32_t _rapidos = 0;
volatile uint32_t _tPrimV = 0;
volatile uint32_t _tUltV = 0;
volatile uint32_t _per[N_PER];
volatile uint8_t  _nPer = 0;

// Resultados
float    _f = 0.0f;
float    _q = 0.0f;
float    _vol = 0.0f;
float    _coherencia = 0.0f;
float    _tiempoSinPulso_s = 0.0f;
bool     _fallo = false;

uint32_t _vFlancos = 0, _vValidos = 0, _vGlitch = 0, _vRapidos = 0;
int32_t  _vPcnt = -1;
uint32_t _accValidos = 0, _accGlitch = 0, _accVentanasRech = 0;
uint32_t _tUltLogRech_ms = 0;

#if CAUDAL_USAR_PCNT_DIAG
pcnt_unit_handle_t _pcntUnit = nullptr;
pcnt_channel_handle_t _pcntChan = nullptr;
int _pcntPrev = 0;
bool _pcntOk = false;
static constexpr int PCNT_HIGH = 30000;
#endif

portMUX_TYPE _mux = portMUX_INITIALIZER_UNLOCKED;
};

```

caudalimetro.cpp

```

#include "caudalimetro.h"
#include "soc/gpio_reg.h"

// -----
// Parámetros por defecto
// Alimentación: hasta ~666 Hz. LOW/HIGH mínimos 250 µs (el HIGH se

```

```

acorta por la RC lenta).
// Permeado:      hasta 250 Hz (sobrado para < 200 mL/min). LOW/HIGH
mínimos 800 µs.
// -----
-----
static ParamPulso paramDefecto(bool esAlim) {
    ParamPulso p;
    if (esAlim) {
        p.anchoMinLow_us   = 250;
        p.anchoMinHigh_us  = 250;
        p.periodoMin_us    = 1500;
        p.qMin_mLmin       = 30.0f;
    } else {
        p.anchoMinLow_us   = 800;
        p.anchoMinHigh_us  = 800;
        p.periodoMin_us    = 4000;
        p.qMin_mLmin       = 15.0f;
    }
    p.periodoMax_us        = 700000; // < ~1.4 Hz => detenida
    p.toleranciaMediana    = 0.40f;
    p.fraccionCoherenteMin = 0.70f;
    return p;
}

Caudalimetro::Caudalimetro(uint8_t pin, float k, const char* nombre,
bool esAlimentacion)
    : _pin(pin), _k(k), _nombre(nombre), _esAlimentacion(esAlimentacion),
      _p(paramDefecto(esAlimentacion)) {
    for (uint8_t i = 0; i < N_PER; i++) _per[i] = 0;
}

void Caudalimetro::setParametros(const ParamPulso& p) {
    portENTER_CRITICAL(&_mux);
    _p = p;
    portEXIT_CRITICAL(&_mux);
}

bool IRAM_ATTR Caudalimetro::leerNivel() const {
    if (_pin < 32) return (REG_READ(GPIO_IN_REG) >> _pin) & 0x1;
    return (REG_READ(GPIO_IN1_REG) >> (_pin - 32)) & 0x1;
}

void Caudalimetro::begin() {
    pinMode(_pin, INPUT_PULLUP);

#ifdef CAUDAL_USAR_PCNT_DIAG
    pcnt_unit_config_t ucfg = {};
    ucfg.low_limit = -1000;
    ucfg.high_limit = PCNT_HIGH;
    if (pcnt_new_unit(&ucfg, &_pcntUnit) == ESP_OK) {
        pcnt_glitch_filter_config_t fcfg = {};
        fcfg.max_glitch_ns = 12000; // ~960 ciclos APB (máx. 1023)
        pcnt_chan_config_t ccfg = {};
        ccfg.edge_gpio_num = _pin;
        ccfg.level_gpio_num = -1;
        if (pcnt_unit_set_glitch_filter(_pcntUnit, &fcfg) == ESP_OK &&
            pcnt_new_channel(_pcntUnit, &ccfg, &_pcntChan) == ESP_OK &&
            pcnt_channel_set_edge_action(_pcntChan, PCNT_CHANNEL_EDGE_ACTION_TO_ZERO)) {

```

```

        pcnt_channel_set_edge_action(_pcntChan,
        PCNT_CHANNEL_EDGE_ACTION_HOLD,          // flanco de subida:
no cuenta
        PCNT_CHANNEL_EDGE_ACTION_INCREASE)      // flanco de bajada:
+1
        == ESP_OK &&
        pcnt_channel_set_level_action(_pcntChan,
        PCNT_CHANNEL_LEVEL_ACTION_KEEP,
PCNT_CHANNEL_LEVEL_ACTION_KEEP) == ESP_OK &&
        pcnt_unit_enable(_pcntUnit) == ESP_OK &&
        pcnt_unit_clear_count(_pcntUnit) == ESP_OK &&
        pcnt_unit_start(_pcntUnit) == ESP_OK) {
        _pcntOk = true;
        _pcntPrev = 0;
    }
}
    if (!_pcntOk) Serial.printf("[%s] PCNT no disponible: diagnóstico en
silicio desactivado\n", _nombre);
#endif

    uint32_t t = micros();
    _nivelAlto = leerNivel();
    _tSubida = t;
    _tBajada = t;
    _bajadaCandidata = false;
    _hayUltValido = false;

    attachInterruptArg(digitalPinToInterrupt(_pin), isrPuente, this,
CHANGE);
}

void IRAM_ATTR Caudalimetro::isrPuente(void* arg) {
    reinterpret_cast<Caudalimetro*>(arg)->isr();
}

// -----
// Máquina de estados por nivel real del pin:
// HIGH -> LOW : candidato si el HIGH previo duró >= anchoMinHigh
// LOW -> HIGH : pulso válido si fue candidato y el LOW duró >=
anchoMinLow,
// y además el periodo respecto del último válido es >=
periodoMin
// La marca temporal del pulso es el flanco de BAJADA (rápido, no lo
afecta la RC).
// -----
void IRAM_ATTR Caudalimetro::isr() {
    uint32_t t = micros();
    bool alto = leerNivel();

    portENTER_CRITICAL_ISR(&_mux);
    _flancos++;

    if (!alto) {
        if (_nivelAlto) {
            _bajadaCandidata = ((t - _tSubida) >= _p.anchoMinHigh_us);
            if (!_bajadaCandidata) _glitch++;
        }
    }
}

```

```

        _tBajada = t;
        _nivelAlto = false;
    } else {
        _glitch++; // par de flancos más corto que la latencia de la ISR
    }
} else {
    if (!_nivelAlto) {
        uint32_t anchoBajo = t - _tBajada;
        if (_bajadaCandidata && anchoBajo >= _p.anchoMinLow_us) {
            uint32_t tp = _tBajada;
            bool aceptar = true;
            if (_hayUltValido) {
                uint32_t per = tp - _tUltValido;
                if (per < _p.periodoMin_us) {
                    aceptar = false;
                    _rapidos++;
                } else if (per <= _p.periodoMax_us && _nPer < N_PER) {
                    _per[_nPer++] = per;
                }
            }
            if (aceptar) {
                if (_validos == 0) _tPrimV = tp;
                _tUltV = tp;
                _validos++;
                _tUltValido = tp;
                _hayUltValido = true;
            }
        } else if (_bajadaCandidata) {
            _glitch++;
        }
        _bajadaCandidata = false;
        _tSubida = t;
        _nivelAlto = true;
    } else {
        _glitch++;
    }
}
portEXIT_CRITICAL_ISR(&_mux);
}

uint32_t Caudalimetro::mediana(uint32_t* v, uint8_t n) {
    for (uint8_t i = 1; i < n; i++) {
        uint32_t x = v[i];
        int8_t j = (int8_t)i - 1;
        while (j >= 0 && v[j] > x) { v[j + 1] = v[j]; j--; }
        v[j + 1] = x;
    }
    return (n & 1) ? v[n / 2] : (v[n / 2 - 1] / 2 + v[n / 2] / 2);
}

void Caudalimetro::actualizar(float dt_s, bool bombaEmpuja) {
    uint32_t per[N_PER];
    uint8_t nPer;
    uint32_t n, flancos, glitch, rapidos, tPrim, tUlt, tUltValido;
    bool hayUlt;

    portENTER_CRITICAL(&_mux);
    // ... (rest of the function body) ...
}

```

```

    n = _validos; flancos = _flancos; glitch = _glitch; rapidos =
    _rapidos;
    tPrim = _tPrimV; tUlt = _tUltV; nPer = _nPer;
    tUltValido = _tUltValido; hayUlt = _hayUltValido;
    for (uint8_t i = 0; i < nPer; i++) per[i] = _per[i];
    _validos = 0; _flancos = 0; _glitch = 0; _rapidos = 0; _nPer = 0;
    portEXIT_CRITICAL(&_mux);

    _vFlancos = flancos; _vValidos = n; _vGlitch = glitch; _vRapidos =
    rapidos;
    _accGlitch += glitch;

#if CAUDAL_USAR_PCNT_DIAG
    if (_pcntOk) {
        int cnt = 0;
        if (pcnt_unit_get_count(_pcntUnit, &cnt) == ESP_OK) {
            int d = cnt - _pcntPrev;
            if (d < 0) d += PCNT_HIGH; // al llegar a high_limit el contador
vuelve a 0
            _pcntPrev = cnt;
            _vPcnt = d;
        }
    }
#endif

    // 1. Estimación de frecuencia robusta (mediana + coherencia +
consistencia con el span)
    float f = 0.0f;
    bool ventanaOk = false;
    _coherencia = 0.0f;

    if (n >= 3 && nPer >= 2 && tUlt > tPrim) {
        uint32_t med = mediana(per, nPer); // per queda ordenado
        uint8_t dentro = 0;
        float tol = _p.toleranciaMediana * (float)med;
        for (uint8_t i = 0; i < nPer; i++) {
            if (fabsf((float)per[i] - (float)med) <= tol) dentro++;
        }
        _coherencia = (float)dentro / (float)nPer;

        float fMed = 1000000.0f / (float)med;
        float fSpan = ((float)(n - 1) * 1000000.0f) / (float)(tUlt -
tPrim);
        bool consistente = fabsf(fSpan - fMed) <= 0.25f * fMed;

        if (_coherencia >= _p.fraccionCoherenteMin && consistente) {
            f = fSpan;
            ventanaOk = true;
        }
    }

    // 2. Turbina detenida si el último pulso válido es demasiado antiguo
    if (hayUlt && (uint32_t)(micros() - tUltValido) > _p.periodoMax_us) {
        f = 0.0f;
        if (n < 3) ventanaOk = false;
    }

    if (n >= 3 && !ventanaOk) {

```

```

    _accVentanasRech++;
    uint32_t ahora = millis();
    if (ahora - _tUltLogRech_ms >= 5000) {
        _tUltLogRech_ms = ahora;
        Serial.printf("[%s] Ventana rechazada: validos=%lu coh=%.2f
glitch=%lu rapidos=%lu flancos=%lu\n",
                        _nombre, (unsigned long)n, _coherencia, (unsigned
long)glitch,
                        (unsigned long)rapidos, (unsigned long)flancos);
    }
}

// 3. Caudal y límites físicos
float q = ventanaOk ? (f * 1000.0f) / _k : 0.0f;
if (q < _p.qMin_mLmin) { q = 0.0f; ventanaOk = false; }
if (q > Q_MAX_FISICO_MLMIN) {
    q = 0.0f;
    ventanaOk = false;
    _accVentanasRech++;
}
_f = ventanaOk ? f : 0.0f;

// 4. Volumen: solo pulsos validados en ventanas aceptadas
if (ventanaOk) {
    _vol += (float)n / (_k * 60.0f);
    _accValidos += n;
}

// 5. EMA (tau ≈ 4.5 s)
if (q > 0.0f) {
    _q = (_q == 0.0f) ? q : (0.20f * q + 0.80f * _q);
} else {
    _q *= 0.50f;
    if (_q < 1.0f) _q = 0.0f;
}

// 6. Pérdida de señal: solo los pulsos VALIDADOS resetean el fallo
(el ruido ya no lo enmascara)
if (ventanaOk) {
    _tiempoSinPulso_s = 0.0f;
    _fallo = false;
} else if (bombaEmpuja && _esAlimentacion) {
    _tiempoSinPulso_s += dt_s;
    if (_tiempoSinPulso_s >= 5.0f) _fallo = true;
} else {
    _tiempoSinPulso_s = 0.0f;
    _fallo = false;
}
}
}

```

Integración opcional en el .ino (telemetría para la prueba en seco)

Esta línea reemplaza el `printf` de `[TELEMETRIA]`:

```

Serial.print(F("[TELEMETRIA] RPM: %d, f: %d, q: %f, coh: %.2f, glitch: %lu, rapidos: %lu, flancos: %lu, vol: %f, accValidos: %lu, accRech: %lu, tiempoSinPulso: %f, fallo: %d",

```

```
Serial.printf("[TELEMETRIA] RPM: %4.1f | %s | Q_Alim: %5.1f (val %lu / brut %lu / pcnt %ld / gl %lu) | "
              "Q_Perm: %5.1f (val %lu / brut %lu / pcnt %ld / gl %lu) | V_Perm: %.4f L | Rech_P: %lu\n",
              bomba.rpmActual(), bomba.enRegimenEstable() ? "ESTABLE" : "RAMPA",
              qAlim, (unsigned long)sensorAlimentacion.pulsosValidos(),
              (unsigned long)sensorAlimentacion.pulsosBrutos(),
              (long)sensorAlimentacion.pcntVentana(),
              (unsigned long)sensorAlimentacion.glitchesVentana(),
              qPerm, (unsigned long)sensorPermeado.pulsosValidos(),
              (unsigned long)sensorPermeado.pulsosBrutos(),
              (long)sensorPermeado.pcntVentana(),
              (unsigned long)sensorPermeado.glitchesVentana(),
              sensorPermeado.volumen_L(),
              (unsigned long)sensorPermeado.ventanasRechazadas());
```

Cómo leer la prueba en seco

Lectura en permeado seco	Diagnóstico
brutos > 0, glitches > 0, válidos = 0	El firmware resuelve: el ruido es corto. Certificable con la prueba húmeda.
brutos > 0, pcnt ≈ 0	Glitches menores a 12 µs; el silicio los filtra solo.
válidos > 0 de forma sostenida	Ruido con LOW > 0,8 ms (masa o nivel). Barrera física: hay que tocar el cableado.
En húmedo, pcnt ≈ 2 × válidos	Confirmado el doble conteo por la rampa RC; el K antiguo era ≈2× el real.

Pendiente de banco: no lo pude compilar ni probar en un ESP32 en esta revisión. Lo verificaría primero en dos puntos:

Que la API de PCNT de su versión de core 3.x acepte la configuración. Si falla, el código lo `#define CAUDAL_USAR_PCNT_DIAG 0`

Que la prueba húmeda a 20 mL/min no pierda pulsos con el ancho mínimo de 800 µs.